

TF-IDF + Cosine Similarity Baseline — Model Summary

Milestone 1: NLP Foundation & Semantic Similarity

Approach

A retrieval-style baseline with no learned classifier:

- 1. Clean text (collapse whitespace) for `prompt` and options `A`–`E`.
- 2. Fit a single `TfidfVectorizer` (unigrams + bigrams, English stopwords removed, `min_df=2`) on all prompt + option text from the **training split only**.
- 3. Vectorize each `prompt` and each of its 5 options.
- 4. Compute **cosine similarity** between the prompt vector and each option vector.
- 5. Rank the 5 options by similarity score; the **top 3** become the MAP@3 prediction.
- 6. Softmax over the 5 similarity scores gives a pseudo-"confidence" for visualization (not a calibrated probability).

No model weights are learned — the "model" is entirely the fitted TF-IDF vocabulary + IDF weights, saved as `tfidf_baseline_model.joblib`.

Configuration

Parameter	Value
<code>lowercase</code>	True
<code>stop_words</code>	english
<code>ngram_range</code>	(1, 2)
<code>min_df</code>	2
<code>test_size</code>	0.2 (stratified by answer label)
<code>random_state</code>	42

Results (80/20 stratified validation split)

Metric	Score	Reference
Top-1 Accuracy	0.158	Random baseline: 0.20

Metric	Score	Reference
Macro F1	0.154	—
MAP@3 (the actual Kaggle metric)	0.280	Random baseline: ≈0.367

This baseline performs worse than random guessing on both metrics. It is not fit for submission as-is.

Root Cause Analysis

Diagnosed directly from the validation similarity scores, not assumed:

	Mean cosine similarity
Correct option vs. prompt	0.174
Incorrect option vs. prompt	0.183

The two are statistically indistinguishable (correct is even marginally *lower*), which means prompt↔option lexical overlap carries almost no signal for this dataset.

Why: the 5 options per question are **near-paraphrases of each other**, not independent distractors. Checking option-to-option similarity within a question:

Example question	Option-to-option cosine similarity range
Q1284	0.72 - 0.74 (all 5 options nearly identical in wording)
Q1213	0.71 - 0.91
Q88 (more distinct options)	0.12 - 0.32

Distractors are written to be textually very close to the correct answer (e.g. all 5 Heidegger options open with "Martin Heidegger believes..." and share most of the sentence structure). TF-IDF measures **lexical overlap with the prompt**, but correctness here depends on **factual/semantic accuracy within an already-shared vocabulary** — a distinction pure TF-IDF cosine similarity cannot make.

A secondary issue: a handful of validation rows return **0.0 similarity across all 5 options**, where `(stop_words="english") + (min_df=2)` strip every word shared between a short prompt and its options, leaving no overlap to score at all.

From earlier EDA, "always pick the longest option" alone scores **34.9% top-1 accuracy** — already ahead of this TF-IDF baseline's 15.8%. This confirms the failure is in the *similarity* signal specifically, not a lack of any signal in the data.

Conclusions for the Project Report

1. **TF-IDF prompt↔option cosine similarity is not a viable standalone signal** for this dataset, because distractors are deliberately close paraphrases of the correct answer rather than unrelated text.
2. A pure retrieval/similarity approach conflates "shares words with the question" with "is factually correct," which don't align when every option shares most of its wording with the others.
3. **Next steps to explore** (candidates for Milestone 2-4 per the project's progression toward transformers/fine-tuning):
 - Supervised classification on prompt+option pairs (e.g. logistic regression / gradient boosting over TF-IDF features) instead of unsupervised similarity ranking.
 - Distinctiveness features — how different an option is from the *other 4 options*, not just from the prompt.
 - Move to contextual embeddings (BERT/RoBERTa via `sentence-transformers`) per Milestone 2, since dense semantic embeddings can capture meaning differences between near-paraphrases that sparse TF-IDF cannot.
4. This baseline is still useful to keep in the report as a **documented negative result** — it demonstrates understanding of *why* a naive approach fails on this specific dataset, which is exactly the kind of error analysis the L1/L2 viva rubric asks for.

Model artifact: `tfidf_baseline_model.joblib` (fitted `TfidfVectorizer`). Script: `03_tfidf_baseline.py`. Logged to Weights & Biases project `23f3003691-t22026`, run `TFIDF_Baseline_Complete`.